

A DISTRIBUTED SIMULATION SANDBOX FOR DECENTRALIZED CONTROL ALGORITHMS IN MARINE DRONE SWARMS

ALAN TOMASZ BEJNAROWICZ 4132829

Masters Project in Engineering (Drones and Autonomous Systems)
Supervisor: Xiaofeng Xiong

March 21, 2026



The Maersk Mc-Kinney Moeller Institute
University of Southern Denmark

Abstract

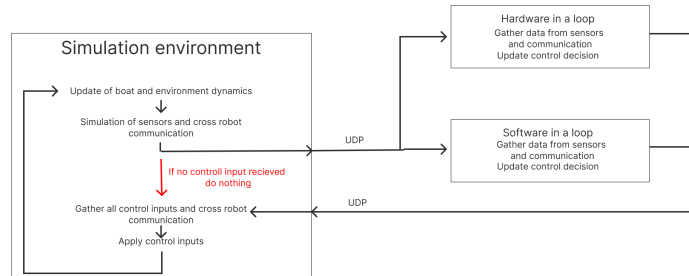
This thesis proposes the design and implementation of a high-performance simulation sandbox for developing and testing decentralized control algorithms for autonomous sailing drone swarms. The core of the project is a simulation environment. The simulator models key marine dynamics, including wind and sea conditions, and incorporates virtual sensors, along with inter-drone communication channels.

The motivation for this work is its application in maritime search and rescue (SAR). By leveraging a swarm of inexpensive, autonomous drones, the time to locate a drowning victim can be significantly reduced, dramatically increasing the chances of survival.

As of October 2025, there are no known existing solutions that combine simulation, SITL/HITL capabilities, and decentralized control for marine drone swarms, making this project a pioneering effort in autonomous maritime robotics. Existing tools tend to target aerial drones or are general-purpose robotics simulators and require heavy modification to support marine dynamics and large-scale swarm experiments. A related project is YARA[4], a Gazebo-based sailing drone simulator, but it lacks SITL/HITL support and is not optimized for extensive swarm simulations.

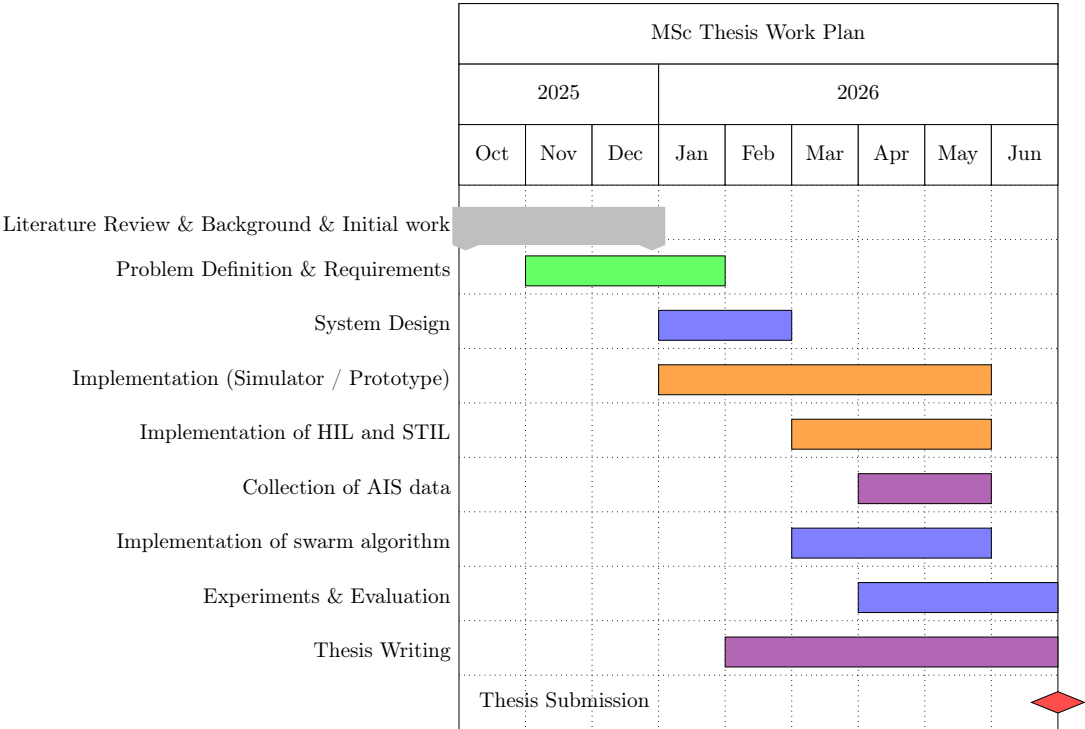
This work will implement a cross-robot communication channel model with signal attenuation, delays, and packet dropouts. The simulator will provide sensor models for IMU, GPS, and AIS (Automatic Identification System) and will support replaying historical AIS datasets to populate and simulate real vessel traffic within the environment. The platform will also allow isolating and instantiating independent swarms to enable large-scale training and evaluation of genetic algorithms and neural-network based controllers.

The simulator will be validated through a series of experiments, including benchmarking decentralized control algorithms for formation keeping, collision avoidance, and cooperative search tasks. The performance of the simulator will be assessed based on its computational efficiency, scalability, and fidelity in replicating real-world marine conditions.



Proposed simulation dataflow.

WORK SCHEDULE / GANTT CHART



0.1 Drone Ship Physics Model

Very important part of my project is to have a good physics model of the drone ship. That model will be used to simulate IMU and GPS sensor readings, as well as to provide a realistic environment for testing control algorithms.

First step is to model the buoyancy of the ship. The ship will be modeled as a rigid body with a certain mass, width, length, and draft. Currently model does not assume any particular shape for the ship, but it will be important to consider the shape in future iterations to improve the accuracy of the model. The buoyancy force will be calculated based on the 4 points of the ship: the bow, stern, port, and starboard. The buoyancy force will be calculated as the weight of the displaced water, which is equal to the volume of the submerged part of the ship multiplied by the density of water and gravity.

Same points will be used to calculate the wave drag force. In each of 4 points there is a wave drag force acting in the normal direction to the surface of the water. Forces are scaled to the size of the ship.

This approach captures the essential physics of the problem while keeping the model computationally efficient. Placement of these points is visible in the figure below 0.1.

That model will be used to simulate the motion of the ship in response to control inputs and environmental forces.

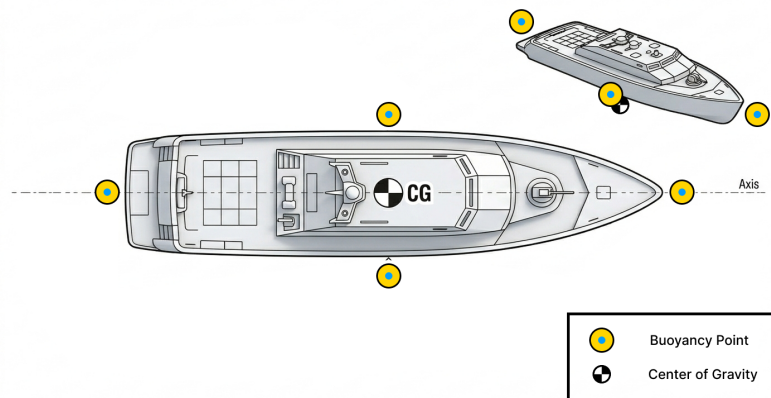


Figure 0.1: Ship buoyancy model diagram.

Introduction

The physics engine (implemented in ShipBuoyancy.cs script) utilizes a multi-point "floater" system to calculate the upward force of displacement, wave drag force, coupled with linear damping and keel-drag curves to simulate water resistance.

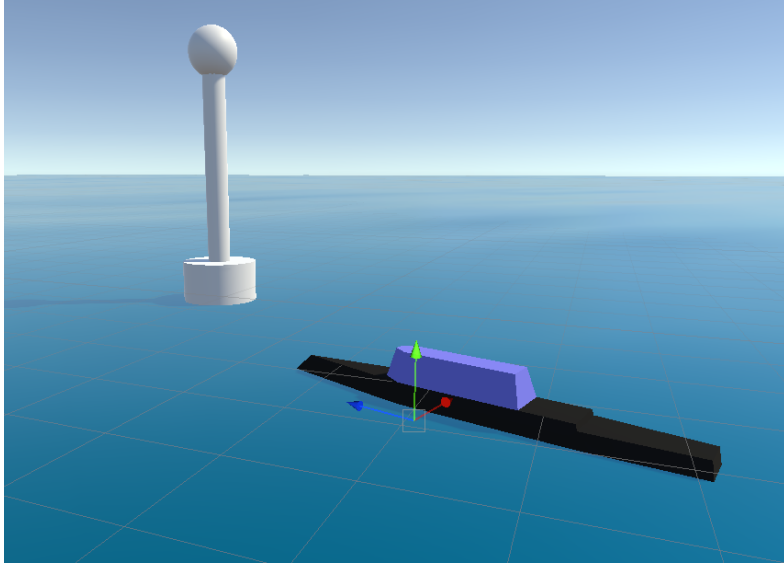


Figure 0.2: Left floater position in simulated ship. $\hat{Z}$ axis (blue) is forward, $\hat{X}$ axis (red) is right and $\hat{Y}$ (green) is up.

Model of Buoyancy

According to Archimedes' Principle, the buoyancy force F_b is equal to the weight of the displaced fluid. In this implementation, we approximate the submerged volume by checking the depth h of N discrete points (floaters) relative to the water surface y_{sea} . For each floater i , the submergence factor s_i is defined as:

$$s_i = \text{clamp}_{01} \left(\frac{y_{sea} - y_i}{h_{max}} \right) \quad (1)$$

The total buoyancy force $\mathbf{F}_B$ applied at the floater position is:

$$\mathbf{F}_B = \sum_{i=1}^n (k_b \cdot s_i) \hat{Y} \quad (2)$$

where k_b is the buoyancyForce constant and $\hat{Y}$ is the world-up vector.

Damping and Stability

To prevent harmonic oscillation (infinite bouncing), a damping force $\mathbf{F}_d$ is applied based on the vertical velocity v_y at the floater's specific world-space position:

$$\mathbf{F}_d = -(v_y \cdot \zeta \cdot s_i) \hat{Y} \quad (3)$$

where ζ represents the buoyancyDamper.

Global Water Resistance

When the ship is submerged ($s_i > 0$), global drag tensors are applied to the RigidBody to simulate fluid viscosity:

$$\mathbf{F}_{drag} = -\mathbf{v} \cdot C_d \quad (4)$$

$$\tau_{drag} = -\omega \cdot C_a \quad (5)$$

where C_d is waterDrag and C_a is waterAngularDrag.

The Keel Effect (Lateral Resistance)

Real ships have keels that allow forward movement but resist sideways "slipping." This is modeled using a slip angle θ , derived from the dot product of the planar velocity $\mathbf{v}_p$ and the ship's forward vector $\hat{\mathbf{Z}}$.

$$\cos(\theta) = \frac{\mathbf{v}_p \cdot \hat{\mathbf{Z}}}{\|\mathbf{v}_p\|} \quad (6)$$

The resulting sideways force is:

$$\mathbf{F}_{keel} = -(\mathbf{v} \cdot \hat{\mathbf{X}}) \cdot \text{DragFunction}(1 - |\cos \theta|) \cdot k_{keel} \quad (7)$$

where $\hat{\mathbf{X}}$ is right vector, k_{keel} is a coefficient. DragFunction is set by user in spline form visible in 0.3.

Hydrostatic Pressure Gradient and Wave-Slope Dynamics

To simulate the "surfing" effect where a vessel reacts to the slope of a swell, we incorporate a force derived from the surface normal $\hat{\mathbf{n}}$. Unlike static buoyancy, which only acts along the world-up axis $\hat{\mathbf{Y}}$, the pressure gradient in a moving fluid creates a lateral force component.

The "Wave-Slide" force $\mathbf{F}_s$ is modeled by projecting the gravitational vector $\mathbf{g}$ onto the tangent plane of the water surface. This represents the component of gravity that pulls the vessel "downhill" along the face of a wave:

$$\mathbf{F}_s = -(\mathbf{g} - (\mathbf{g} \cdot \hat{\mathbf{n}})\hat{\mathbf{n}}) \cdot k_{surf} \cdot s_i \quad (8)$$

Where: $\hat{\mathbf{n}}$ is the unit surface normal at the floater position. Variable k_{surf} is a coefficient, and s_i is the submergence ratio of the floater ($0 \leq s_i \leq 1$) taken from (1).

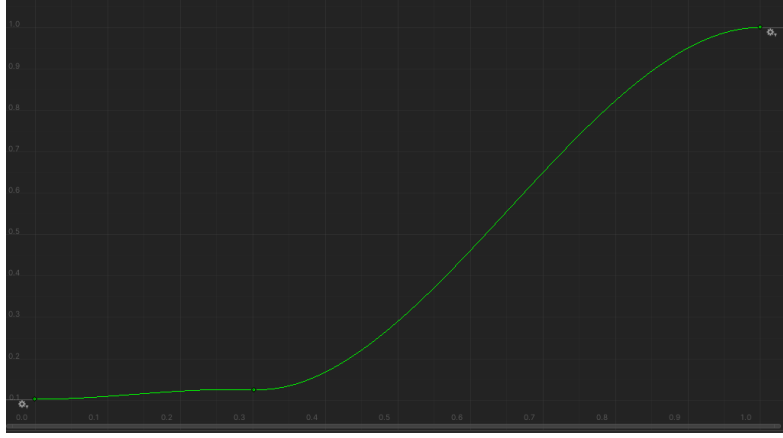


Figure 0.3: Keel drag curve represents water friction in function of slip angle of moving ship. Drag is low when slip angle is close to 0 deg and high when approaching 90 deg

Center of Mass / Gravity (CG) Configuration

Stability is determined by the relationship between the Center of Buoyancy (average of submerged floaters) and the Center of Mass. By lowering the CoM below the geometric center, a restoring torque is naturally generated by the physics engine, ensuring the ship remains upright. In a model ship used in simulation CG is set to be below hull. That makes the drone "unsinkable" which is desired in small sea drones. On large ships CG is usually much higher to prevent large stabilizing forces. If a massive ship had a CG as low as a small drone, the "Righting Moment" would be so powerful that the sudden, violent snapping back to center could cause structural fatigue or catastrophic failure in heavy seas.

Propulsion Dynamics

The propulsion system in the simulation assumes a simplified thrust model where the force F_{prop} is directly proportional to the throttle input $\delta_t \in [-1, 1]$ and the defined motor power P_m .

$$\mathbf{F}_{prop} = \delta_t \cdot P_m \cdot \hat{\mathbf{n}}_{prop} \quad (9)$$

where $\hat{\mathbf{n}}_{prop}$ represents the forward unit vector of the propeller mount.

The Propeller Wash Effect (Slipstream)

In real-world ships, the rudder is often positioned directly behind the screw to take advantage of the accelerated flow. Simulator models the "Effective Flow Velocity" V_{eff} at the rudder as:

$$V_{eff} = v_z + \left(\delta_t \cdot \frac{P_m}{m} \cdot \eta_{wash} \right) \quad (10)$$

Where: v_z is the forward local velocity of the ship. m is the mass of the vessel. η_{wash} is the effect coefficient. This ensures that the vessel maintains maneuverability even at low hull speeds, provided the engine is engaged.

Hydrodynamic Forces on the Rudder

The rudder is treated as an airfoil generating two primary force components: Lift (the turning force) and Drag (the resistive force).

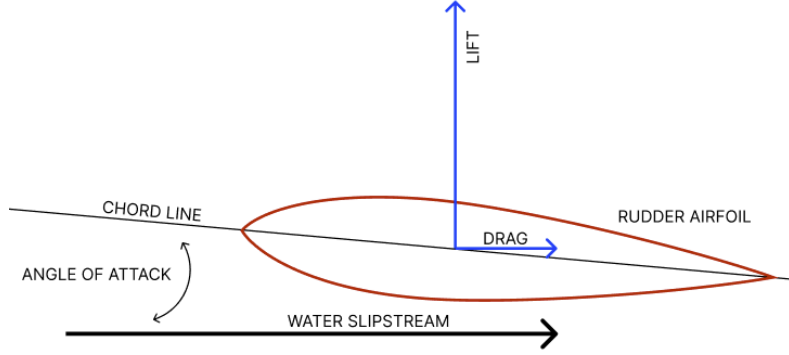


Figure 0.4: Rudder airfoil model.

The lift magnitude L is calculated using a variation of the fluid lift equation, where the force is proportional to the square of the velocity and the sine of the rudder angle α .

$$L = V_{eff}^2 \cdot C_L \cdot \sin(\alpha) \cdot \text{sgn}(V_{eff}) \quad (11)$$

where: C_L is lift coefficient representing the rudder's effectiveness, α rudder angle in radians.

The act of turning the rudder introduces a drag force D that acts against the forward motion of the ship, simulating the energy loss during a turn.

$$D = V_{eff}^2 \cdot C_D \cdot |\sin(\alpha)| \cdot \text{sgn}(V_{eff}) \quad (12)$$

where C_D is the dragCoefficient.

The final force applied to the ship at the rudder's position $\mathbf{p}_{rudder}$ is the vector sum of the lift and drag:

$$\mathbf{F}_{total} = (L \cdot \hat{\mathbf{R}}\mathbf{r}) + (-D \cdot \hat{\mathbf{R}}\mathbf{f}) \quad (13)$$

Where $\hat{\mathbf{R}}\mathbf{r}$ is the rudder's right-facing unit vector and $\hat{\mathbf{R}}\mathbf{f}$ is the ship's forward unit vector. Because this force is applied at a distance d from the Center of Mass, it generates the turning moment required to rotate the ship.

Summary of Forces and Moments

The dynamic behavior of the drone ship is governed by the summation of all hydrodynamic, gravitational, and propulsion forces acting upon the rigid body. To ensure a coherent physics simulation, these forces are categorized into restorative, resistive, and active control components. The total force $\mathbf{F}_{total}$ and total torque τ_{total} acting at the Center of Mass (CoM) can be summarized by the following Newtonian equations:

$$\mathbf{F}_{total} = \mathbf{F}_g + \sum_{i=1}^n (\mathbf{F}_{B,i} + \mathbf{F}_{d,i} + \mathbf{F}_{s,i}) + \mathbf{F}_{drag} + \mathbf{F}_{keel} + \mathbf{F}_{prop} + \mathbf{F}_{rudder} \quad (14)$$

$$\tau_{total} = \sum_{i=1}^n (\mathbf{r}_i \times \mathbf{F}_{net,i}) + (\mathbf{r}_{prop} \times \mathbf{F}_{prop}) + (\mathbf{r}_{rud} \times \mathbf{F}_{rudder}) + \tau_{drag} \quad (15)$$

Where $\mathbf{r}$ denotes the lever arm vector from the CoM to the specific point of force application.

Force Classification Matrix The interactions described in this model are synthesized in Table 0.1, illustrating the physical origin and the intended simulation effect of each component.

Table 0.1: Summary of Forces in the Drone Ship Physics Model

Force Component	Symbol	Type	Primary Physical Function
Buoyancy	$\mathbf{F}_B$	Restorative	Counteracts gravity and provides static stability.
Buoyancy Damping	$\mathbf{F}_d$	Dissipative	Prevents non-physical oscillations at the interface.
Wave-Slide (Slope)	$\mathbf{F}_s$	Environmental	Simulates pressure gradients on wave faces.
Global Drag	$\mathbf{F}_{drag}$	Resistive	Simulates skin friction and form drag of the hull.
Keel Resistance	$\mathbf{F}_{keel}$	Constraining	Provides lateral stiffness, minimizing leeway.
Propulsion Thrust	$\mathbf{F}_{prop}$	Active	Provides forward acceleration.
Rudder Lift/Drag	$\mathbf{F}_{rudder}$	Control	Generates turning moment and induced drag.

0.2 Wave Physics Model

The sea surface height $\eta(x, z, t)$ is modeled as a superposition of multiple sinusoidal components, mimicking a simplified Gerstner Wave and following Pierson-Moskowitz spectrum. The total displacement at any point $\mathbf{p}$ is the sum of n wave harmonics:

$$\eta(\mathbf{p}, t) = \sum_{i=1}^n A_i \cdot \sin(\mathbf{k}_i \cdot \mathbf{p}_x + \omega_i t) \cdot \cos(\mathbf{k}_{i,z} \cdot \mathbf{p}_z \cdot \phi + \omega_i t) \quad (16)$$

Where: A_i is the amplitude of the i -th harmonic. $\mathbf{k}_i$ is the wave vector (determining frequency and direction). ω_i is the angular frequency (phase speed). ϕ is a directional interference constant (set to 0.8 in this model to avoid perfect symmetry). The implementation categorizes these into three distinct scales: Primary Swells: Low frequency, high amplitude (e.g., $f = 0.02$, $A = 0.8$). Turbulent Mid-levels: Moderate frequency for sea-state texture. Capillary Ripples: High frequency, low amplitude for surface detail. Values used in simulation are visible in table 0.2.

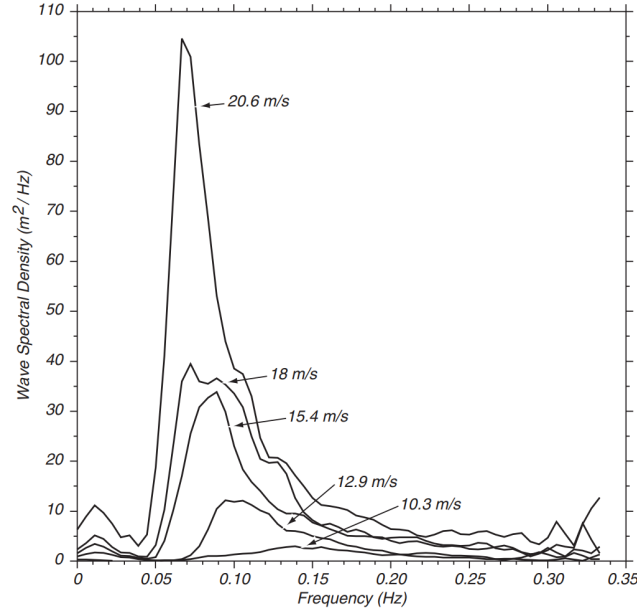


Figure 0.5: Wave spectra for different wind speeds according to Moskowitz (1964). [2]

Table 0.2: Spectral Parameters for Procedural Wave Synthesis

Wave Category	Frequency (f)	Speed (v)	Amplitude (A)
Primary Swell 1	0.02 Hz	1.2 m/s	0.80 m
Primary Swell 2	0.04 Hz	1.5 m/s	0.50 m
Mid-level Turbulence	0.10 Hz	2.2 m/s	0.20 m
Secondary Turbulence	0.18 Hz	1.8 m/s	0.15 m
Surface Micro-Ripple	0.45 Hz	3.0 m/s	0.05 m
Capillary Ripple	0.80 Hz	4.2 m/s	0.02 m

To calculate buoyancy forces, the surface normal $\hat{\mathbf{n}}$ must be derived. Rather than analytical differentiation, which becomes computationally expensive as n increases, a Finite Difference Method is used. By sampling the height at four orthogonal offsets around a point $\mathbf{p}$, the slope is determined:

$$\frac{\partial \eta}{\partial x} \approx \frac{\eta(x - \epsilon, z) - \eta(x + \epsilon, z)}{2\epsilon} \quad (17)$$

$$\frac{\partial \eta}{\partial z} \approx \frac{\eta(x, z - \epsilon) - \eta(x, z + \epsilon)}{2\epsilon} \quad (18)$$

The resulting normal vector is:

$$\hat{\mathbf{n}} = \text{normalize} \left(\frac{\partial \eta}{\partial x}, 1.0, \frac{\partial \eta}{\partial z} \right) \quad (19)$$

This is later used in Physics Model to calculate Wave-Slope Dynamics.

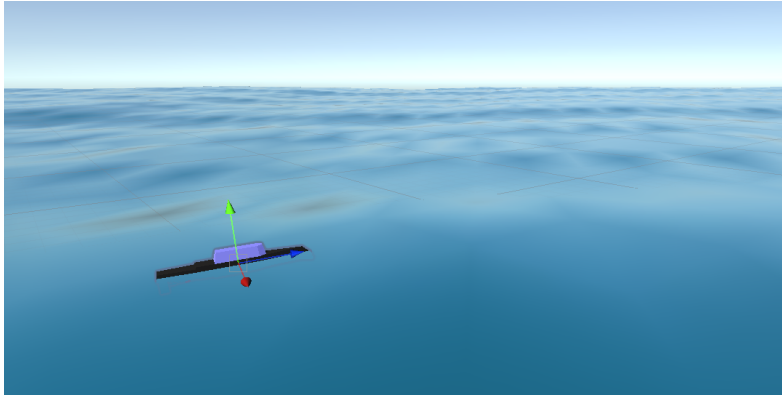


Figure 0.6: Waves simulation results.

Introduction to Sensor Simulation

To bridge the gap between a high-fidelity physics environment and autonomous control logic, the simulation must provide synthetic data that replicates the characteristics of physical hardware. In a real-world marine deployment, the vessel's State Estimation is derived from a suite of diverse sensors, each with its own coordinate frame, sampling rate, and error profile. The virtual sensor suite developed for this simulator focuses on three primary components:

1. **Global Positioning System (GPS):** Provides low-frequency absolute positioning and Ground Speed.
2. **Inertial Measurement Unit (IMU):** A high-frequency 6-axis sensor measuring proper linear acceleration ($\mathbf{a}$) and angular velocity (ω).

3. **Digital Compass/Magnetometer:** Provides a heading reference (ψ) relative to the magnetic North.

The following sections detail the mathematical transformation of "Ground Truth" data—the perfect state vectors provided by the physics engine—into noisy, biased, and discretized signals. This approach ensures that control algorithms, such as PID loops or Extended Kalman Filters (EKF), are tested against the same signal degradation they would encounter at sea.

Global Positioning System (GPS) Simulation

The GPS module simulates a receiver providing absolute geographic coordinates (ϕ, λ) . Because the simulation environment utilizes a flat Cartesian grid, a conversion between Euclidean distance and Geodetic displacement is required. A reference origin, GPS_{ref} , is defined with a known Latitude ϕ_0 and Longitude λ_0 . The current position is derived by projecting the ship's relative displacement $(\Delta x, \Delta z)$ onto the Earth's ellipsoidal surface (WGS84). The change in latitude $\Delta\phi$ is calculated as:

$$\Delta\phi = \frac{\Delta z}{R_M} \quad (20)$$

where R_M is the meridian radius of curvature ($\approx 111,111$ m per degree). The change in longitude $\Delta\lambda$ depends on the current latitude, as the meridians converge toward the poles:

$$\Delta\lambda = \frac{\Delta x}{R_E \cos(\phi_0)} \quad (21)$$

where R_E is the Earth's equatorial radius. The final simulated coordinates incorporate a lower sampling frequency ($f_s = 5-10$ Hz) and horizontal dilution of precision (HDOP) modeled as Gaussian noise η_{gps} :

$$\phi_{sim} = \phi_0 + \Delta\phi + \eta_\phi \quad (22)$$

$$\lambda_{sim} = \lambda_0 + \Delta\lambda + \eta_\lambda \quad (23)$$

Inertial Measurement Unit (IMU) Simulation

The virtual IMU (Inertial Measurement Unit) is implemented to provide high-frequency kinematic data, replicating a 6-axis MEMS (Micro-Electro-Mechanical Systems) sensor. To maintain high fidelity for industrial control applications, the sensor is sampled at a frequency of $f_s = 200$ Hz within the physics integration loop.

$$\mathbf{a}_{sf,world} = \frac{\mathbf{v}t - \mathbf{v}t - \Delta t}{\Delta t} - \mathbf{g} \quad (24)$$

where $\mathbf{g} = [0, -9.81, 0]^T$ represents the gravity vector. The world-space specific force is then transformed into the sensor's local coordinate system $\mathcal{B}$ using the inverse rotation transformation:

$$\mathbf{a}_{local} = \mathbf{R}^{-1} \cdot \mathbf{a}_{sf, world} \quad (25)$$

The gyroscope readings are derived directly from the rigid body's angular velocity vector ω . To align with the physical sensor orientation, the world-space angular velocity is projected onto the local axes of the sensor mount:

$$\omega_{local} = \mathbf{R}^{-1} \cdot \omega_{world} \quad (26)$$

To simulate electronic interference and mechanical vibrations, the raw signals are augmented with White Gaussian Noise. The noise is generated using the Box-Muller transform, which converts uniformly distributed random variables $u_1, u_2 \sim \mathcal{U}(0, 1)$ into a normal distribution $\mathcal{N}(\mu, \sigma^2)$:

$$z_0 = \sqrt{-2 \ln(u_1)} \cos(2\pi u_2) \quad (27)$$

The final simulated readings for acceleration $\mathbf{y}_a$ and rotation $\mathbf{y}_\omega$ are:

$$\mathbf{y}_a = \mathbf{a}_{local} + \eta_a, \quad \eta_a \sim \mathcal{N}(0, \sigma_a^2) \quad \mathbf{y}_\omega = \omega_{local} + \eta_\omega, \quad \eta_\omega \sim \mathcal{N}(0, \sigma_\omega^2) \quad (28)$$

where σ_a and σ_ω are the standard deviation parameters defined in the sensor configuration.

Digital Compass (Magnetometer)

The heading ψ is derived from the projection of the vessel's longitudinal unit vector $\hat{\mathbf{z}}_{body}$ onto the global horizontal XZ -plane. Assuming the magnetic North aligns with the global $+Z$ axis, the raw heading is given by:

$$\psi_{raw} = \text{atan2}(z_{body,x}, z_{body,z}) \quad (29)$$

To account for sensor imperfections in a marine environment, the simulated reading ψ_{sim} incorporates additive white Gaussian noise η_m to represent electromagnetic interference from the ship's motors:

$$\psi_{sim} = (\psi_{raw} + \eta_m) \pmod{360^\circ} \quad (30)$$

Bibliography

- [1] AISHub. aishub, 2025. Accessed: 2025-12-17.
- [2] J. Henrique G. M. Alves, Michael L. Banner, and Ian R. Young. Revisiting the Pierson–Moskowitz asymptotic limits for fully developed wind waves. *Journal of Physical Oceanography*, 33(7):1301–1323, July 2003.
- [3] OpenSeaMap. Openseamap — free nautical chart and community project, 2025. Accessed: 2025-12-17.
- [4] Eduardo Charles Vasconcellos, Álvaro Pinto Fernandes Negreiros, André Paulo Dantas de Araújo, Raphael Guerra, Philippe Preux, Davi Henrique dos Santos, Luiz Marcos Garcia Gonçalves, and Esteban Walter Gonzalez Clua. Yara: An ocean virtual environment for research and development of autonomous sailing robots and other unmanned surface vessels. *Journal of Intelligent & Robotic Systems*, 111(3):78, 2025.